

Design Patterns in Go Cheatsheet

Lessons: [28](#) — [Creational](#) · [29](#) — [Structural](#) · [30](#) — [Behavioral](#)

Examples: [28](#) · [29](#) · [30](#)

Covers: the GoF catalog translated into idiomatic Go — functional options, embedding, first-class functions

Legend: [*] = pattern or API the lessons have not covered yet

CREATIONAL: the useful zero value

```
var b bytes.Buffer          usable immediately – no constructor at all
var mu sync.Mutex          same
type Config struct{ Timeout time.Duration }    zero = "no timeout", document it
if c.Timeout == 0 { c.Timeout = defaultTimeout }    normalize on use
(the most Go-like creational pattern is not needing a constructor)
```

CREATIONAL: constructor

```
func New(db *sql.DB) *Service { return &Service{db: db} }
func New(...) (*Service, error)    when construction can fail
NewX when the package has several types; New when it has one
validate in the constructor so an existing value is always valid
return a struct, not an interface let the caller decide what it needs
(no constructor overloading in Go – use options instead)
```

CREATIONAL: functional options

```
type Option func(*Server)
func WithTimeout(d time.Duration) Option { return func(s *Server){ s.timeout = d } }
func WithLogger(l *slog.Logger) Option { return func(s *Server){ s.log = l } }
func New(addr string, opts ...Option) *Server {
    s := &Server{addr: addr, timeout: 30 * time.Second}    defaults first
    for _, o := range opts { o(s) }                      then overrides
    return s
}
New(":8080", WithTimeout(5*time.Second))
why                optional params, compatible growth, self-documenting
when not to        2 fields and no future – a config struct is simpler
type Option interface{...} [*] the interface variant, for options that can fail
```

CREATIONAL: the rest

Builder	step-by-step construction; strings.Builder is the model b.Add(x).Add(y).Build() – return the receiver to chain
Factory	func(kind string) (Storer, error) – a switch returning different implementations behind one interface
Registry	map[string]func() Storer + a Register() called from init – new kinds plug in without touching the switch
Singleton	sync.Once + a package var; usually DI is the better answer var once sync.Once; once.Do(func(){ inst = &T{} }) sync.OnceValue(f) [*] the one-liner form
Object pool	sync.Pool for buffers – cuts GC pressure; entries can vanish at any GC
Prototype	a Clone() method; remember maps and slices need deep copies
Abstract factory	rare in Go; a struct of constructor funcs does the same job

STRUCTURAL: embedding vs inheritance

<code>type Server struct { *slog.Logger }</code>	embed to promote methods
<code>s.Info("msg")</code>	promoted, no forwarding code
NOT inheritance	no virtual dispatch: the embedded method calls the EMBEDDED type's methods, not your overrides
override by shadowing	define <code>Server.Info</code> to shadow the promoted one
embed an interface	<code>type Repo struct { UserStore }</code> – satisfy it by delegation and override just the method you care about (test fakes)

STRUCTURAL: adapter, decorator, facade

Adapter	make one interface fit another
<code>http.HandlerFunc(f)</code>	THE canonical example: a func becomes a Handler
<code>type readerFunc func([]byte)(int,error) + a Read method</code>	
Decorator / middleware	same interface in, same interface out, behaviour added
<code>func Logging(next http.Handler) http.Handler</code>	
<code>func WithRetry(c Client) Client</code>	
chain them: <code>Logging(Auth(Metrics(h)))</code>	
Facade	one simple type over several subsystems
<code>Service.PlaceOrder</code>	does inventory + payment + email behind one call
Proxy	same interface, controls access
caching proxy, lazy-loading proxy, permission-checking proxy	

STRUCTURAL: composite, bridge, flyweight

Composite	a tree where leaf and branch share an interface
<code>type Node interface { Size() int }</code>	<code>Dir</code> holds <code>[]Node</code> ; <code>File</code> is a leaf
<code>fs.FS</code> , the DOM, and expression trees are all this shape	
Bridge	split abstraction from implementation so both can vary
<code>Notifier(what) x Transport(how)</code>	– compose instead of a class explosion
Flyweight	share immutable data instead of copying it
interned strings, a shared <code>*template.Template</code> , one <code>*slog.Logger</code>	

BEHAVIORAL: functions as strategy & command

Strategy	a func value, not an interface with one method
<code>type Pricer func(Order) Money</code>	
<code>svc := New(WithPricer(flatRate))</code>	
<code>sort.Slice(s, less)</code>	the stdlib's own strategy pattern
Command	a func value (or a struct when it needs data + Undo)
<code>type Command func() error; queue []Command</code>	
undo stack: store the inverse alongside	
Template Method	THE Go trap: embedding does NOT give virtual dispatch.
<code>Base.Run</code> calling <code>s.step()</code>	runs <code>BASE</code> 's step, never the outer type's.
Do it with a func field or an interface parameter instead.	

BEHAVIORAL: observer, state, chain

Observer / pub-sub	a hub: map[Topic][]chan Event, or a slice of callbacks
publish without blocking	buffered channels + a default case (drop slow subscribers)
unsubscribe	return a func to deregister; leaks are the usual bug
State machine	map[State]map[Event]State, or a method per state
transition(cur, ev) (next, error)	– reject illegal transitions loudly
Chain of responsibility	middleware IS this pattern
each link handles it or passes it on	
Mediator	one type coordinating several; the hub in lesson 58
Memento	snapshot + restore; a copy of the value struct

BEHAVIORAL: iterators & visitor

```
range-over-func (Go 1.23+)
iter.Seq[T] = func(yield func(T) bool)
iter.Seq2[K,V] = func(yield func(K, V) bool)
func (t *Tree) All() iter.Seq[int] {
    return func(yield func(int) bool) { t.walk(yield) }
}
for v := range t.All() { ... }           lazy, composable, no allocation
return false from yield                 stops the iteration early
slices.Collect(seq) / slices.Sorted(seq) / maps.Keys(m)    the bridges
Visitor                                a type switch over a closed set of types
switch n := node.(type) { case *Add: ...; case *Lit: ... }
or an interface method per node type when the set is open
```

PATTERN -> GO TRANSLATION TABLE

Singleton	sync.Once, or just dependency injection
Factory Method	a function returning an interface
Abstract Factory	a struct of function fields
Builder	functional options, or a fluent builder type
Prototype	a Clone() method
Adapter	a func type with a method (HandlerFunc)
Decorator	middleware: func(T) T
Facade	a service struct
Proxy	a wrapper implementing the same interface
Composite	an interface + a slice of itself
Strategy	a func value
Command	a func value or a small struct
Observer	channels, or a slice of callbacks
Iterator	range-over-func (iter.Seq)
Template Method	a func field – NEVER embedding
Chain of Responsibility	middleware chain
Visitor	a type switch
State	a map of transitions, or a state interface
Mediator	a hub goroutine owning the state

TRAPS & MEMORIZE

Template Method via embedding	no virtual dispatch – the base always calls itself
options with no defaults	New() returns a half-built object
a factory with one impl	delete it; you have a constructor
interface with one impl	and no test fake – delete it too
singleton everywhere	a global with extra steps; inject instead
decorator order	outermost runs first; recover must be outside logging
observer without unsubscribe	a permanent leak
unbuffered publish	one slow subscriber blocks the publisher forever
sync.Pool without a reset	you hand out dirty objects
deep vs shallow Clone	maps/slices/pointers are shared unless you copy them
patterns for their own sake	Go prefers a function; reach for a struct only when behaviour needs state